Anatomy of a Run

Six views of what happens between a person typing a request and a structured result coming back — the component map, the capability lifecycle, both conversational turns, the guard chain inside a single replay step, and where sensitive data is allowed to travel.

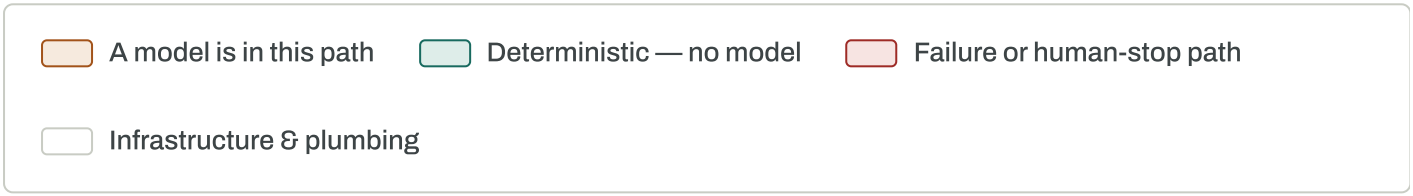
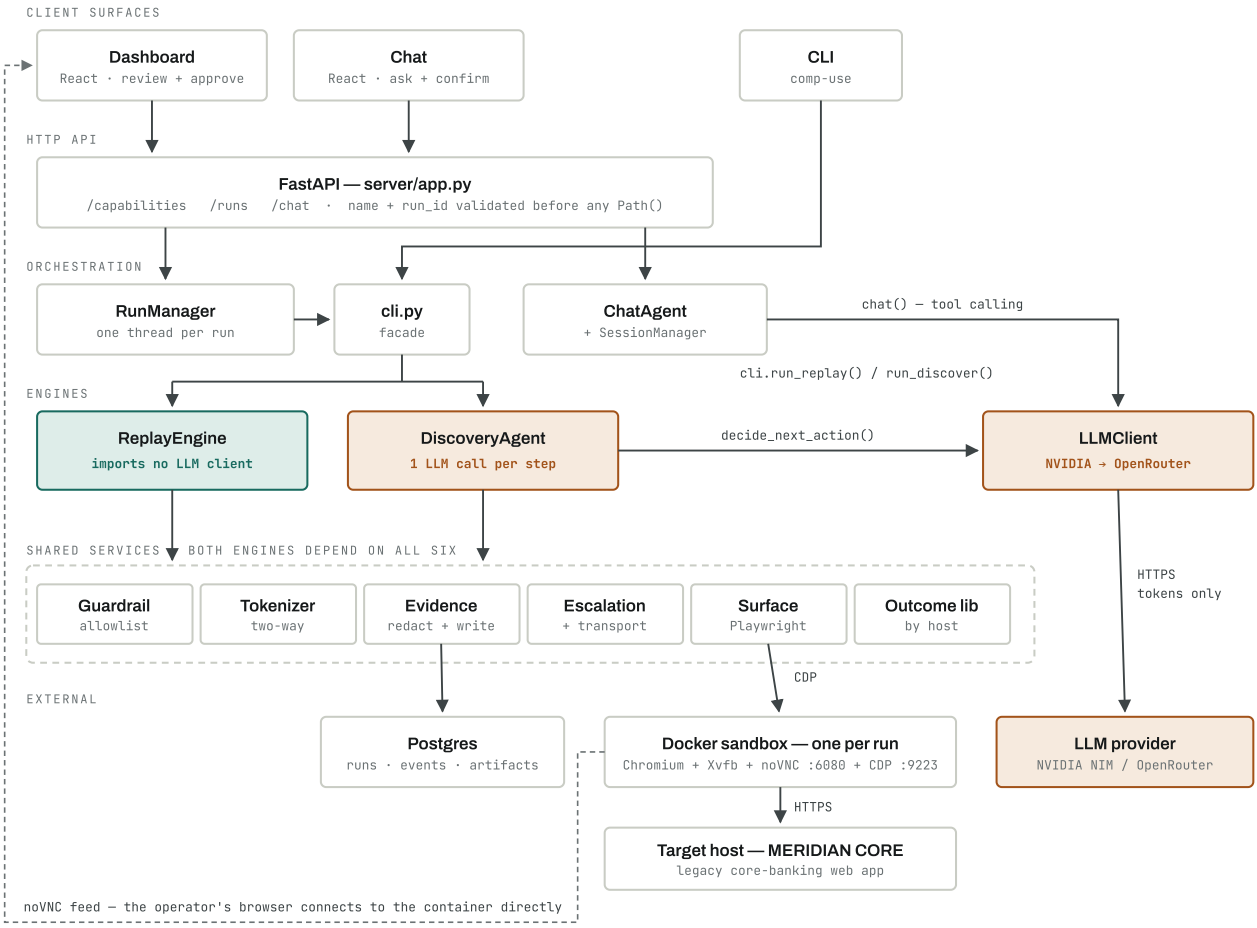


FIG 1 Component map

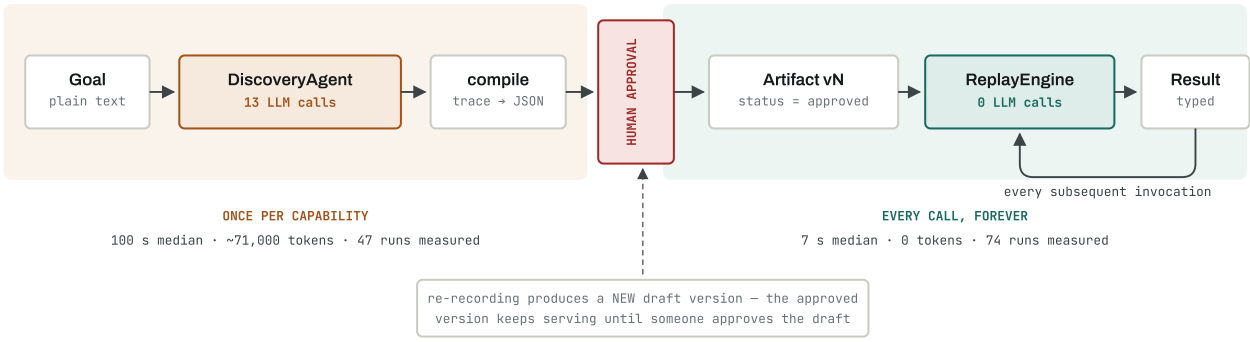
Who calls whom. The claim: exactly one engine touches a model, and the shared services beneath both are the same six.



Read it top-down as a dependency stack. The two engines are siblings on the same shared services, but only `DiscoveryAgent` has an edge to `LLMClient` — `ReplayEngine` has none, and that absence is the architecture's central guarantee. `drift.py` is omitted: it also calls the model, but runs strictly after a replay has produced its final result, so it can't affect one. The dashed line on the left is the video channel: the noVNC feed goes straight from the container to the operator's browser and never passes through FastAPI.

FIG 2 Capability lifecycle

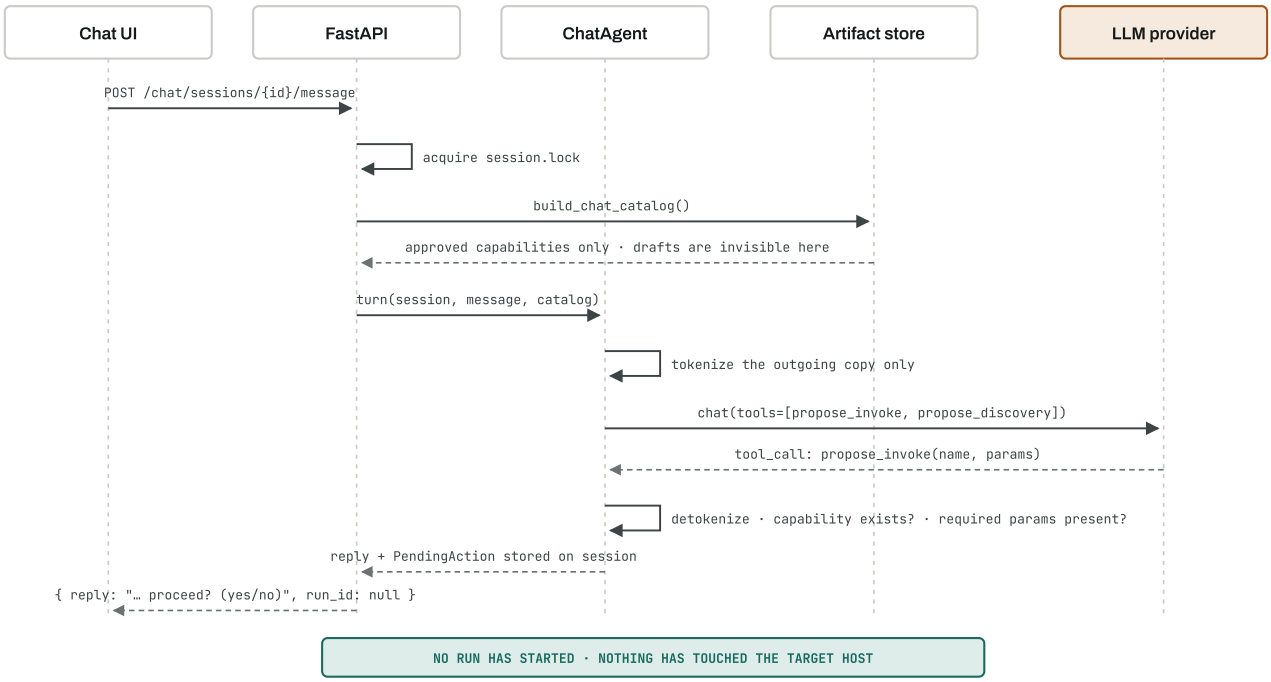
The economic argument in one line: the expensive half runs once, the cheap half runs forever, and a human stands between them.



The approval gate is the only way across. An artifact leaves discovery as a `draft`, and a draft is not invocable — `load_artifact()` with no pinned version returns only approved ones, and a pinned request for a non-approved version is rejected with a 409 before a run is even started. Cost is therefore per *capability*, not per *call*: the orange half is paid once, the teal half is what production actually runs.

FIG 3 Turn one — request becomes a proposal

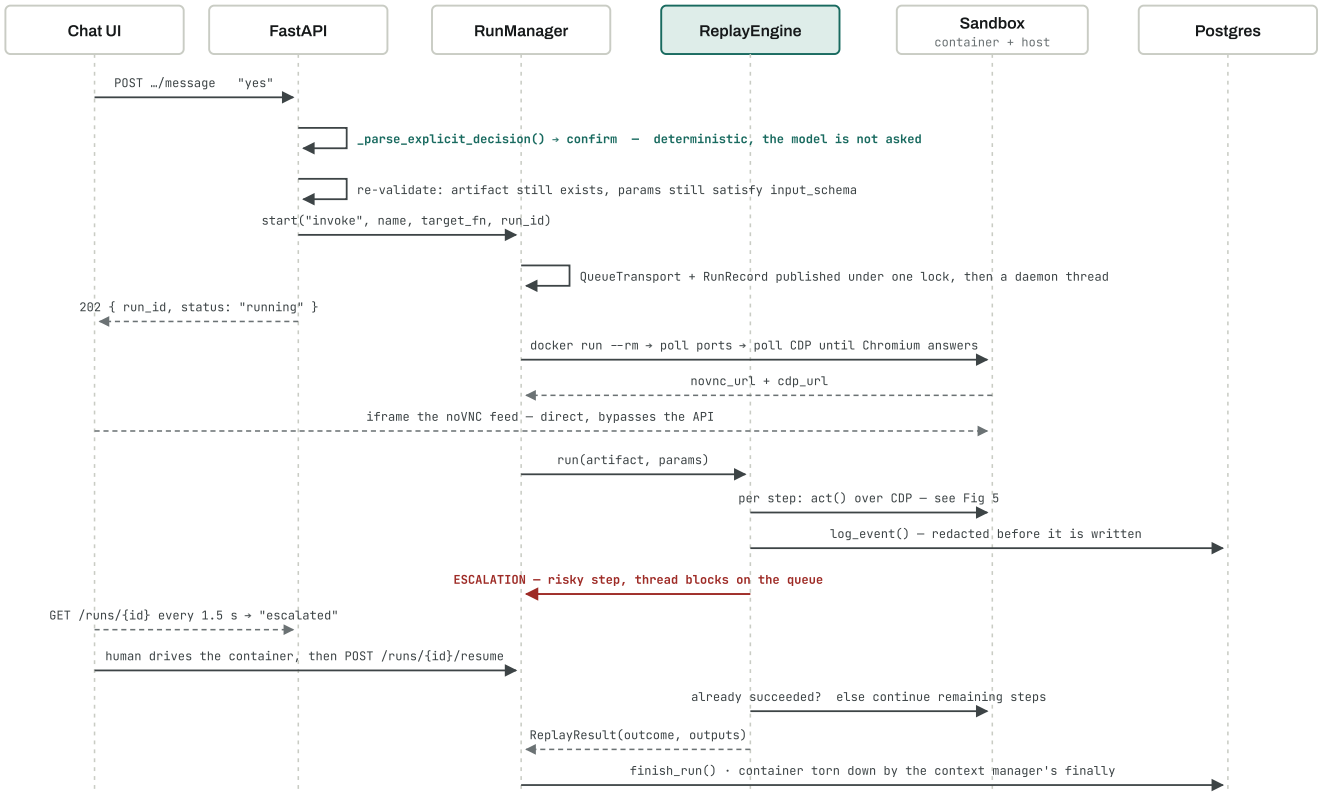
The user asks for something. A model interprets it. Nothing runs.



The model's output is a proposal, not an instruction. Two things bound it: the catalog it reasons over contains only human-approved capabilities, so it cannot propose something nobody signed off; and its proposed parameters are re-checked against the artifact's declared `input_schema` before the user is even asked. The conversation sent to the provider is a tokenized copy — `session.messages` keeps the real text for the app's own display.

FIG 4 Turn two — confirm, run, escalate, resume

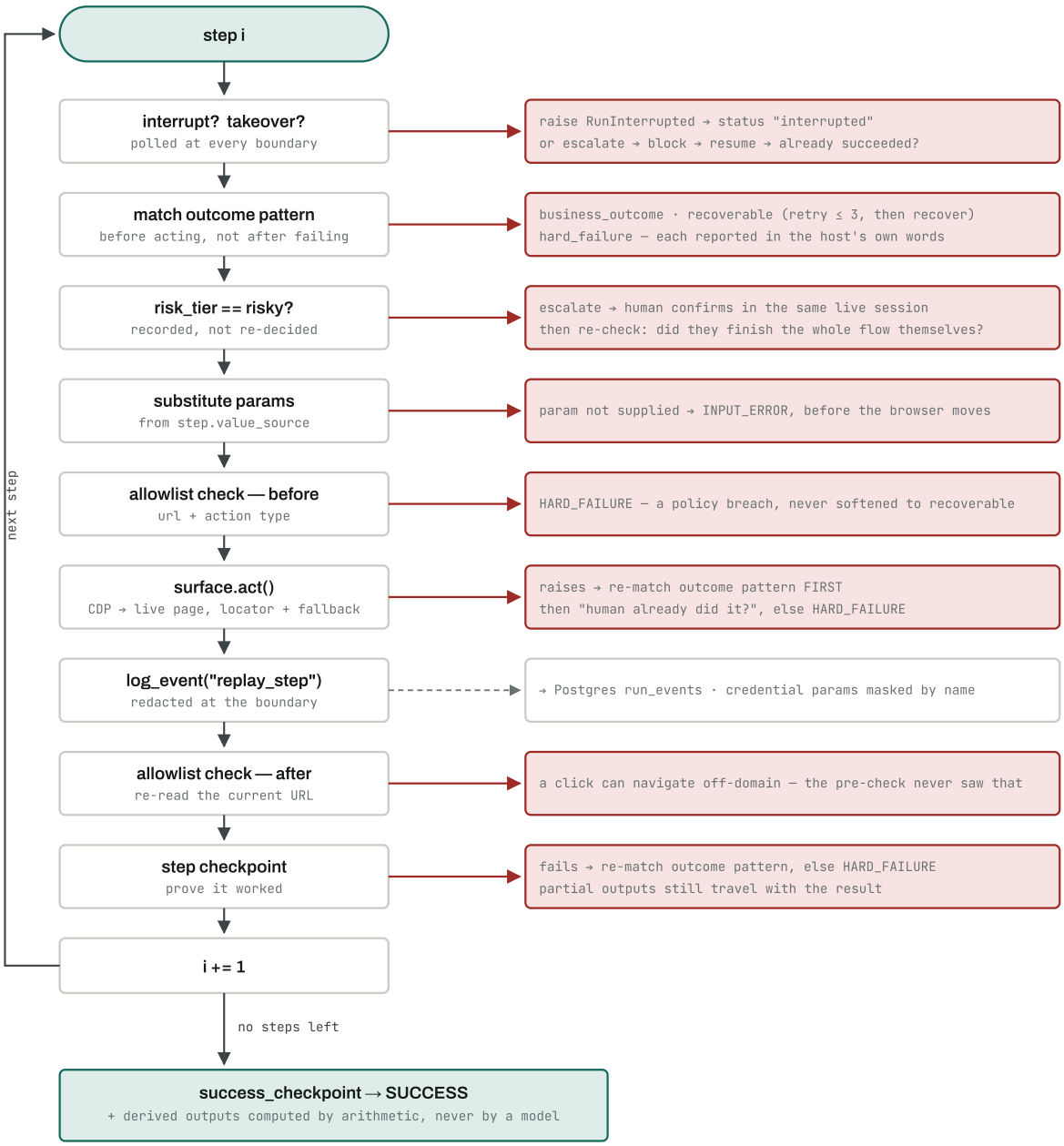
The user says yes. This is the full path from that word to a structured result, including the human handoff.



The confirmation is parsed in code before the model gets a vote. An exact "yes" or "no" resolves deterministically; only an ambiguous reply reaches the model, and a malformed tool call there fails closed to "amend", never to "confirm". Note also that the HTTP request returns at `202` while the browser keeps working — the run outlives the request, which is why the UI polls and why `RunManager` owns a thread rather than the web handler.

FIG 5 Inside one replay step

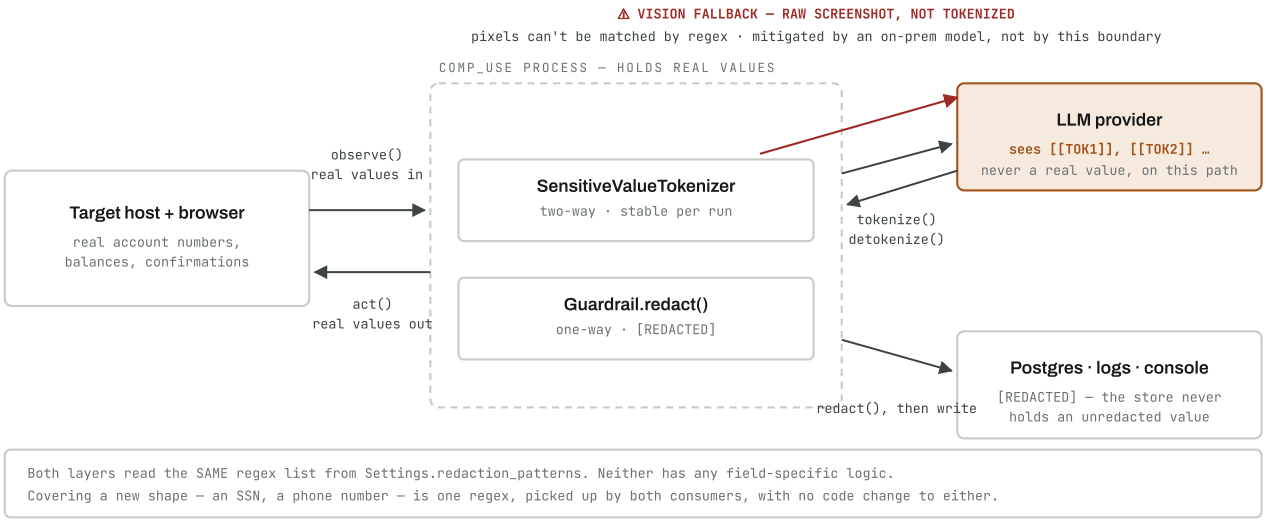
The granular view. Happy path runs straight down; every exception exits right.



Two things to point at. The allowlist is checked twice per step, before and after — because the pre-check validates where the browser *was*, and a click that navigates off-domain would otherwise never be seen. And outcome patterns are matched *before* attempting a step, not only after one fails: if the app has already diverged onto "insufficient funds", the next recorded step targets a control that no longer exists, and blaming the locator would hide the real answer.

FIG 6 Where sensitive data is allowed to go

Three destinations, three different treatments — and one honest hole.



Two mechanisms, one config. Redaction is one-way and protects anything durable; tokenization is two-way and protects the model call, because the model still has to reason about the page while the browser still has to act on real data. The alert-coloured arrow is the gap worth volunteering before anyone finds it: the vision fallback sends a screenshot, and no regex can redact pixels.

Using these in the room

Fig 2 is your opener. It carries the thesis on its own — put it up while you say the "record once, replay forever" line and you won't need to explain it twice.

Fig 1 goes up once and stays available. Don't walk every box. Point at three things: the two engines are siblings, only one has an edge to the model, and everything below the engines is shared. Then move on.

Figs 3 and 4 are the answer to "walk me through a request." The split matters — Fig 3 ends with nothing having happened, which is the safety property. If you only have time for one, use Fig 4 and mention that a proposal turn preceded it.

Fig 5 is your depth card. Hold it back. Bring it out when someone asks how failures are handled or how you know a step actually worked — it answers both, and producing it on demand lands better than presenting it unprompted.

Fig 6 is for the security question, which in a regulated domain is coming whether or not it's on the agenda. Volunteering the vision-fallback gap off this diagram is worth more than any control you could describe instead.